SCHOOL OF BUSINESS AND MANAGEMENT SCIENCES

DEPARTMENT OF FINANCIAL ENGINEERING

PENSION FUND EFFICIENCY TOOLKIT: A PROTOTYPE SYSTEM FOR FRONTIER-BASED OPTIMIZATION ANALYSIS OF SELF-ADMINISTERED PENSION FUNDS IN ZIMBABWE

BY

LORRAINE MUJURU

H220021Y

THIS PROTOTYPE DOCUMENTATION IS SUBMITTED TO THE DEPARTMENT OF FINANCIAL ENGINEERING, SCHOOL OF BUSINESS AND MANAGEMENT SCIENCES, IN PARTIAL FULFILMENT OF THE BACHELOR OF TECHNOLOGY HONOURS DEGREE IN FINANCIAL ENGINEERING

2026

Contents

CHAPTER 1: INTRODUCTION 5

1.1 Background and Motivation 5

1.2 Objectives of the Prototype 5

1.3 Scope 5

CHAPTER 2: SYSTEM ARCHITECTURE 7

2.1 High-Level Architecture 7

2.2 Module Responsibilities 7

2.3 Full Pipeline Flowchart 8

CHAPTER 3: DATA LAYER 9

3.1 Input Data Schema 9

3.2 Dataset Description 9

3.3 DEA Matrix Construction 10

CHAPTER 4: DEA ENGINE 11

4.1 Mathematical Formulation 11

4.1.1 CCR (CRS) Input-Oriented Model 11

4.1.2 BCC (VRS) Extension 11

4.1.3 Two-Phase Slack Maximisation 11

4.2 Scale Efficiency 11

4.3 DEA Solving Flowchart 12

4.4 Efficiency Frontier Diagram 12

4.5 Comparative Efficiency Score Chart 13

CHAPTER 5: SIMAR-WILSON BOOTSTRAP 14

5.1 Motivation 14

5.2 Algorithm 14

5.3 Bootstrap Flowchart 15

CHAPTER 6: RANDOM FOREST SECOND STAGE 16

6.1 Two-Stage Framework 16

6.2 Contextual Features 16

6.3 Model Configuration 16

6.4 Random Forest Pipeline Diagram 17

6.5 Indicative Feature Importance 17

CHAPTER 7: DASHBOARD AND REPORTING 18

7.1 Streamlit Dashboard 18

7.1.1 Tab 1 — Overview 18

7.1.2 Tab 2 — Efficiency 18

7.1.3 Tab 3 — Drivers 18

7.1.4 Tab 4 — Recommendations 19

7.2 PDF Report 19

7.3 Output File Summary 19

CHAPTER 8: TESTING AND DEPLOYMENT 20

8.1 Test Suite 20

8.2 CI/CD Pipeline 20

8.3 CLI Usage 20

8.4 Docker Deployment 20

8.5 Reproducibility Controls 21

CHAPTER 9: RTS AND PEER ANALYSIS 22

9.1 Returns-to-Scale Distribution 22

9.2 Peer Analysis and Benchmarking 22

9.3 Scale Efficiency Distribution 23

CHAPTER 10: CONCLUSION **24**

10.1 Summary of Prototype Contributions **24**

10.2 Limitations and Future Work **24**

10.3 Chapter Summary **25**

CHAPTER 1

Introduction

1.1 Background and Motivation

The Pension Fund Efficiency Toolkit is a software prototype developed in support of the research study *Modeling Efficiency of Self-Administered Pension Funds in Zimbabwe Using a Frontier-Based Optimization Analysis and Random Forest Model*. The study applies Data Envelopment Analysis (DEA) to evaluate the technical efficiency of Zimbabwean self-administered standalone pension funds over the period 2018–2022, and subsequently employs a Random Forest regression model to identify the environmental determinants of efficiency differentials.

The prototype operationalises the full DEA-plus-machine-learning pipeline in Python 3.11, using `PuLP` for linear programming, `scikit-learn` for machine learning, `joblib` for parallel bootstrap computation, `Streamlit` for an interactive dashboard, and `ReportLab` for automated PDF report generation.

1.2 Objectives of the Prototype

1. Implement input-oriented CCR (CRS) DEA to evaluate overall technical efficiency.
2. Implement input-oriented BCC (VRS) DEA to isolate pure technical efficiency.
3. Compute scale efficiency and classify each fund by IRS/CRS/DRS regime.
4. Apply Simar-Wilson (1998) bootstrap bias correction with 95% confidence intervals.
5. Fit a Random Forest second-stage regression on six contextual determinants.
6. Provide a Streamlit dashboard for interactive analysis and PDF export.
7. Support reproducibility through Docker containerisation and GitHub Actions CI.

1.3 Scope

The prototype operates on panel data from 16–20 Zimbabwean self-administered standalone pension funds, each observed over five annual periods (2018–2022).

Data are aggregated to fund-level means before cross-sectional DEA is executed. The system does not currently implement window DEA, Malmquist indices, or stochastic frontier analysis.

CHAPTER 2

System Architecture

2.1 High-Level Architecture

The toolkit is organised as a layered, pipeline-style architecture. Figure 1 shows the high-level module layout; data flows left-to-right through loosely coupled modules.

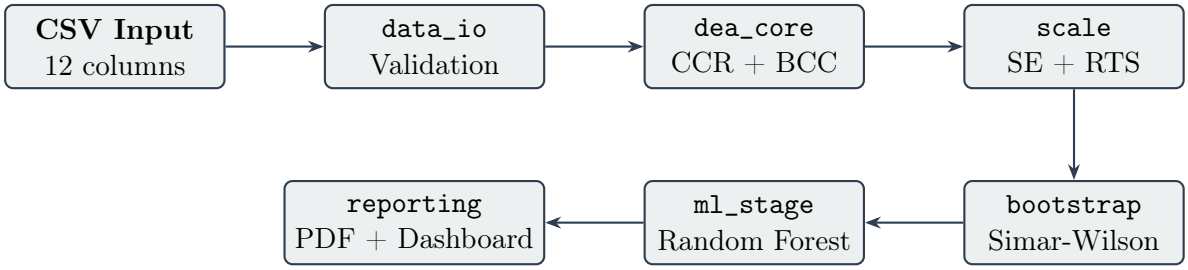

Figure 1: High-level architecture of the Pension Fund Efficiency Toolkit

2.2 Module Responsibilities

Table 1: Toolkit modules and responsibilities

Module	Responsibility
`data_io.py`	CSV loading, schema validation, numeric coercion, missing-value detection, DEA matrix extraction
`dea_core.py`	Two-phase input-oriented LP for CCR and BCC models via PuLP/CBC; peer identification
`scale.py`	Scale efficiency $\theta^{\mathrm{CCR}}/\theta^{\mathrm{BCC}}$; IRS/DRS/CRS classification
`bootstrap.py`	Simar-Wilson Algorithm 1 with Silverman KDE, reflection, joblib parallelism
`ml_stage.py`	Random Forest (500 trees), MDI + permutation importance, CV R^2, PDP plots
`reporting.py`	ReportLab PDF with styled tables and embedded PDP image
`ui_streamlit.py`	4-tab Streamlit dashboard: Overview, Efficiency, Drivers, Recommendations
`cli.py`	Argparse CLI entry point orchestrating the full pipeline

2.3 Full Pipeline Flowchart

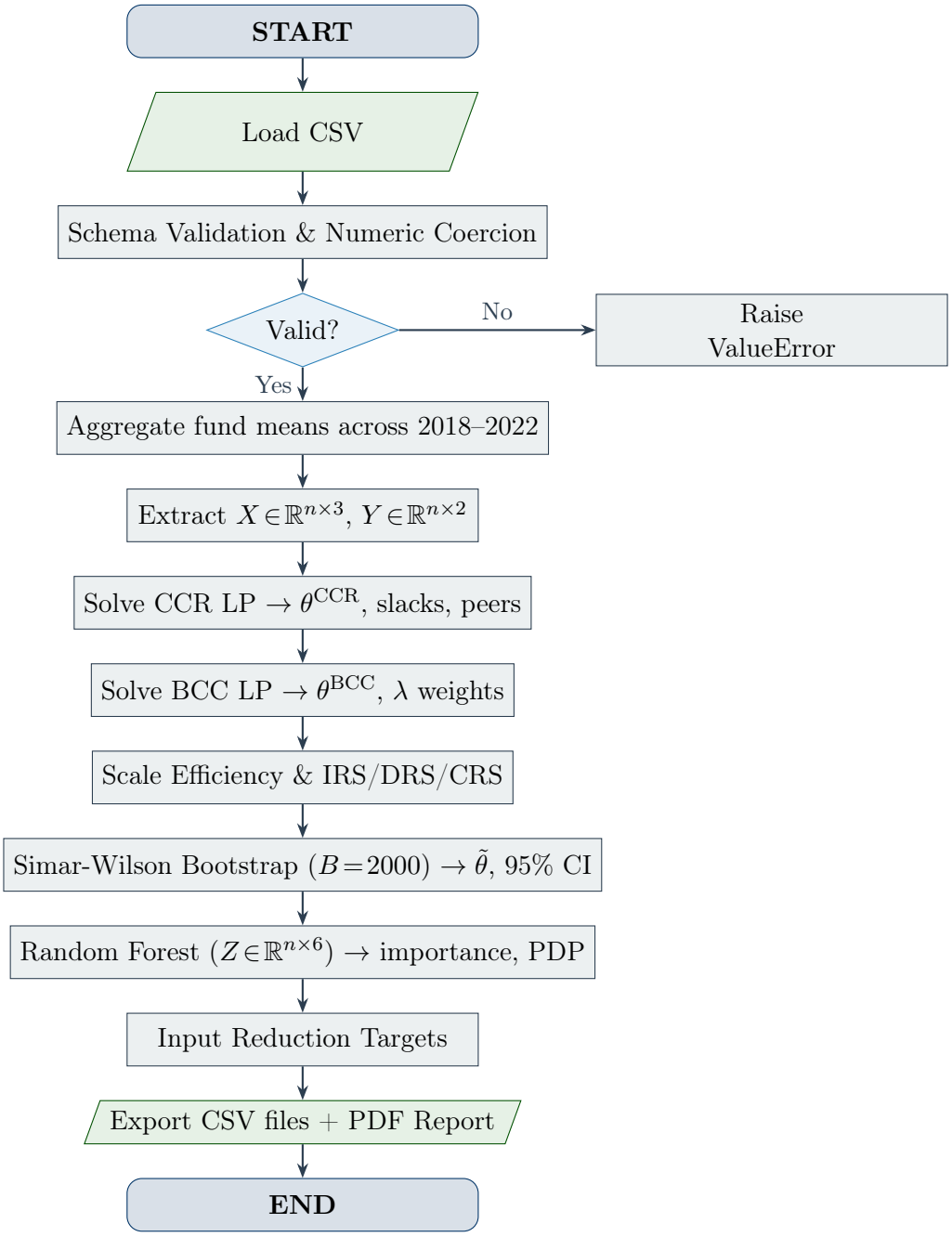

Figure 2: End-to-end pipeline flowchart

CHAPTER 3

Data Layer

3.1 Input Data Schema

The system requires a single CSV with 12 columns. Three columns serve as DEA inputs and two as DEA outputs; the remainder provide contextual metadata.

Table 2: Required CSV schema

Column	Type	Description
`fund_id`	string	Unique identifier (ZW001–ZW020)
`year`	integer	Observation year (2018–2022)
`fund_name`	string	Full fund name
`fund_type`	string	Must be `self_administered`
`total_assets_usd`	float	**DEA Input 1**: total assets (USD)
`operating_expenses_usd`	float	**DEA Input 2**: operating expenses (USD)
`equity_debt_usd`	float	**DEA Input 3**: equity + debt capital (USD)
`net_investment_income_usd`	float	**DEA Output 1**: net investment income (USD)
`member_contributions_usd`	float	**DEA Output 2**: member contributions (USD)
`inflation`	float	Country-level annual inflation rate (%)
`exchange_volatility`	float	Annual exchange rate volatility index
`fund_age`	integer	Years since fund inception

3.2 Dataset Description

The study dataset covers 20 registered Zimbabwean self-administered standalone pension funds (ZW001–ZW020) over 2018–2022 (100 rows total). Fund sizes range from \$7 M to \$584 M in total assets, capturing the scale heterogeneity of the sector. Country-level macroeconomic shocks are reflected in the data: annual inflation reached 557.2% in 2020, and most funds recorded negative net investment income in 2019–2020. Table 3 shows the macroeconomic control variables by year.

Table 3: Macroeconomic control variables by year

Year	**Inflation (%)**	**Exchange Volatility**
2018	10.6	0.18
2019	255.3	0.72
2020	557.2	0.85
2021	98.5	0.45
2022	243.8	0.61

3.3 DEA Matrix Construction

Multi-year panel data are reduced to a cross-section by computing fund-level arithmetic means across 2018–2022. The DEA matrices extracted are:

$$X = \begin{pmatrix} \text{OpEx}_1 & \text{Assets}_1 & \text{Equity}_1 \\ \vdots & \vdots & \vdots \\ \text{OpEx}_n & \text{Assets}_n & \text{Equity}_n \end{pmatrix} \in \mathbb{R}^{n\times 3}, \quad Y = \begin{pmatrix} \text{NetInvInc}_1 & \text{Contrib}_1 \\ \vdots & \vdots \\ \text{NetInvInc}_n & \text{Contrib}_n \end{pmatrix} \in \mathbb{R}^{n\times 2}$$

CHAPTER 4

DEA Engine

4.1 Mathematical Formulation

4.1.1 CCR (CRS) Input-Oriented Model

For each DMU k ($k = 1, \ldots, n$), solve:

$$\begin{aligned} \min_{\theta, \boldsymbol{\lambda}} \ & \theta \\ \text{s.t.} \ & \sum_j \lambda_j x_{ij} \leq \theta x_{ik}, \quad i = 1, \ldots, m \\ & \sum_j \lambda_j y_{rj} \geq y_{rk}, \quad r = 1, \ldots, s \\ & \lambda_j \geq 0 \end{aligned} \tag{1}$$

$\theta \in (0, 1]$ is the radial efficiency score. A score of 1 indicates the fund lies on the efficiency frontier; $1 - \theta$ is the proportional input reduction achievable at the same output levels.

4.1.2 BCC (VRS) Extension

Adding the convexity constraint $\sum_j \lambda_j = 1$ to (1) gives the BCC model, which allows variable returns to scale. The BCC score $\theta^{\text{BCC}} \geq \theta^{\text{CCR}}$.

4.1.3 Two-Phase Slack Maximisation

Phase I solves (1) to obtain θ^*. Phase II fixes $\theta = \theta^*$ and maximises $\sum_i s_i^- + \sum_r s_r^+$ with equality constraints and non-negative slacks, eliminating weak efficiency.

4.2 Scale Efficiency

$$\text{SE}_k = \frac{\theta_k^{\text{CCR}}}{\theta_k^{\text{BCC}}}, \qquad \text{RTS}_k = \begin{cases} \text{IRS} & \sum_j \lambda_j^* < 1 \\ \text{CRS} & \sum_j \lambda_j^* \approx 1 \text{ and SE} \approx 1 \\ \text{DRS} & \sum_j \lambda_j^* > 1 \end{cases} \tag{2}$$

4.3 DEA Solving Flowchart

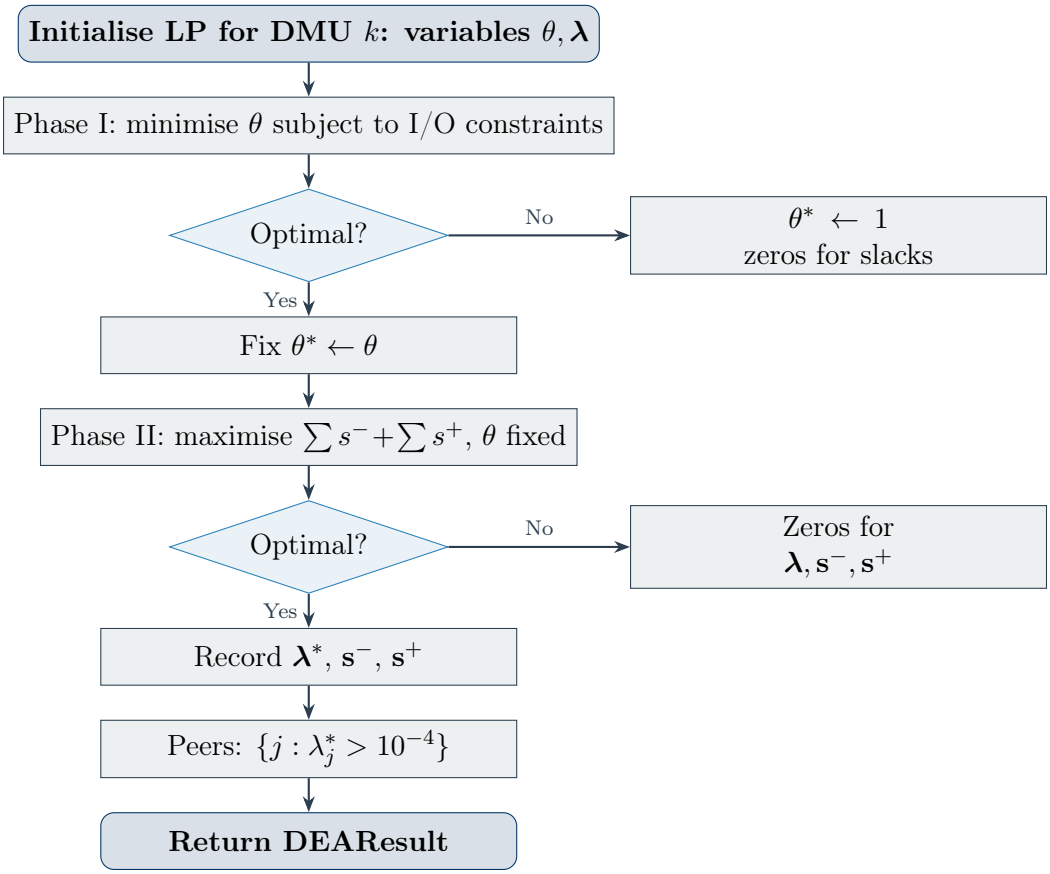

Figure 3: Two-phase DEA LP solving flowchart for a single DMU

4.4 Efficiency Frontier Diagram

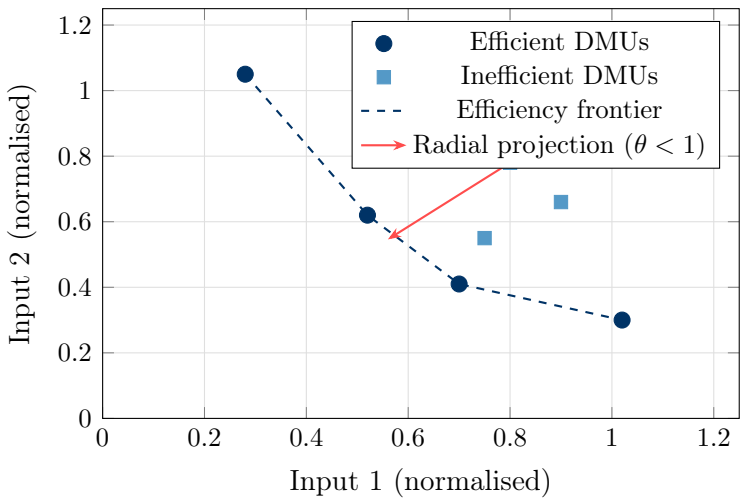

Figure 4: Input-oriented DEA efficiency frontier (conceptual)

4.5 Comparative Efficiency Score Chart

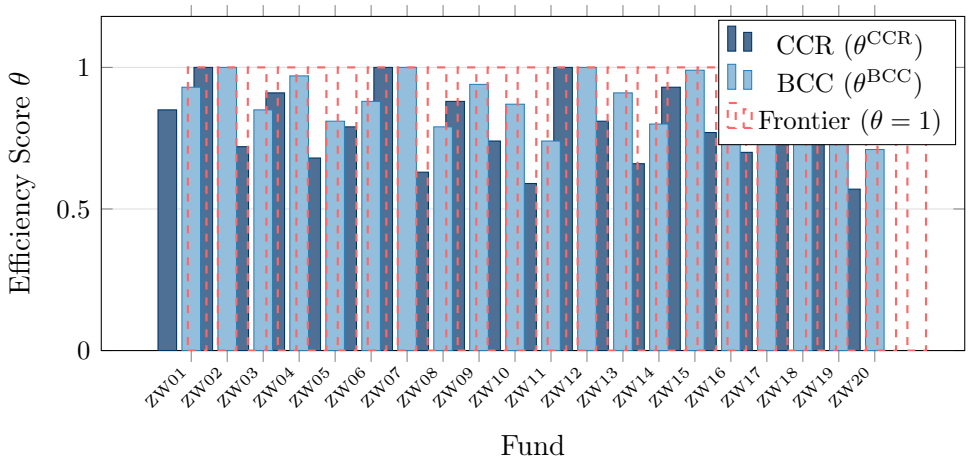

Figure 5: Representative CCR and BCC efficiency scores across 20 pension fund DMUs

CHAPTER 5

Simar-Wilson Bootstrap Bias Correction

5.1 Motivation

DEA scores are upward-biased estimators of true efficiency because the frontier is estimated from the same sample used for evaluation. Simar and Wilson (1998) Algorithm 1 corrects this bias and constructs valid confidence intervals. The implementation in `bootstrap.py` parallelises $B = 2{,}000$ replications with `joblib`.

5.2 Algorithm

1. Compute original DEA scores $\hat{\boldsymbol{\theta}}$.
2. Estimate Silverman bandwidth: $h = 0.9 \min\!\big(\hat{\sigma}_\theta,\ \frac{\mathrm{IQR}_\theta}{1.34}\big) n^{-1/5}$.
3. For $b = 1, \ldots, B$ (parallelised):
   (a) Draw $\boldsymbol{\theta}^*$ from reflected KDE of $\hat{\boldsymbol{\theta}}$; reflect values > 1 as $2 - \theta^*$.
   (b) Pseudo-inputs: $X^* = \mathrm{diag}(\boldsymbol{\theta}^* / \hat{\boldsymbol{\theta}})\, X$.
   (c) Solve DEA on $(X^*, Y) \to \hat{\boldsymbol{\theta}}^{*b}$.
4. Bias: $\hat{B}_k = \overline{\hat{\theta}^*_k} - \hat{\theta}_k$.
5. Bias-corrected: $\tilde{\theta}_k = \hat{\theta}_k - \hat{B}_k$.
6. 95% CI from empirical bootstrap percentiles.

5.3 Bootstrap Flowchart

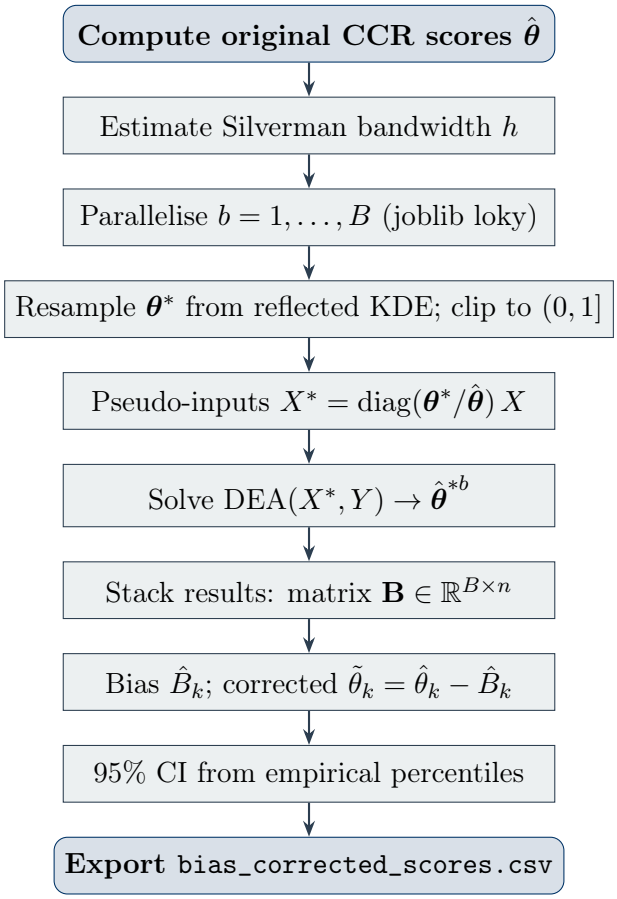

Figure 6: Simar-Wilson bootstrap bias-correction flowchart

CHAPTER 6

Random Forest Second Stage

6.1 Two-Stage Framework

Stage 1 (DEA) produces bias-corrected scores $\tilde{\theta}_k$. Stage 2 regresses these scores on six contextual variables using a Random Forest regressor to quantify which structural and environmental factors drive efficiency differentials.

6.2 Contextual Features

Table 4: Second-stage Random Forest features

Feature	**Source**	**Definition**
`inflation`	CSV	Annual inflation rate (%)
`exchange_volatility`	CSV	Annual exchange rate volatility index
`fund_age`	CSV	Years since inception
`fund_size_log`	Derived	ln(mean total assets)
`expense_ratio`	Derived	mean OpEx / mean total assets
`rts_encoded`	Derived	IRS= +1, CRS= 0, DRS= −1

6.3 Model Configuration

RandomForestRegressor: 500 trees, `max_features="sqrt"`, fixed seed.
Feature importance: MDI (built-in) + permutation importance (30 repeats).
Validation: k-fold cross-validation ($k \leq 5$), reporting mean ± std R^2.
Visualisation: Partial Dependence Plots (PDPs) for top-3 features.

6.4 Random Forest Pipeline Diagram

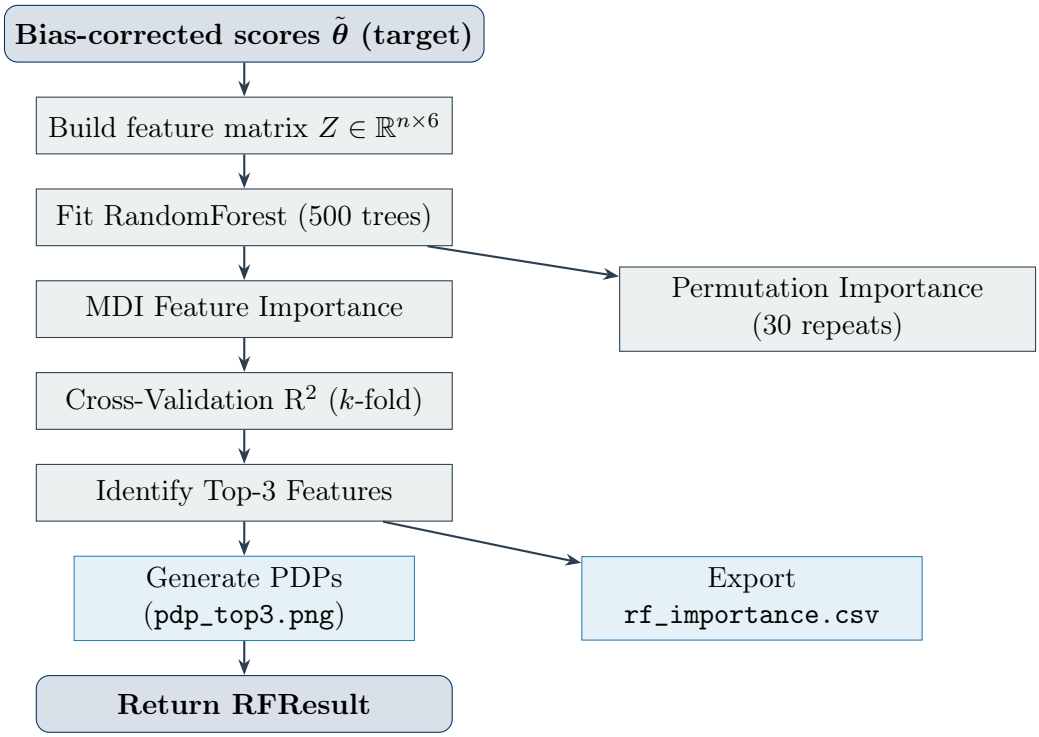

Figure 7: Random Forest second-stage analysis pipeline

6.5 Indicative Feature Importance

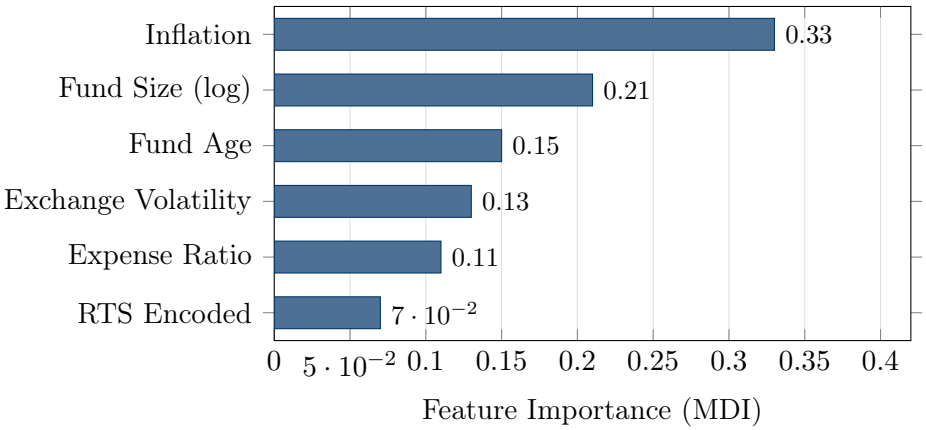

Figure 8: Indicative Random Forest MDI feature importance

CHAPTER 7

Dashboard and Reporting

7.1 Streamlit Dashboard

The interactive dashboard is built with Streamlit (`ui_streamlit.py`) and organised into four tabs.

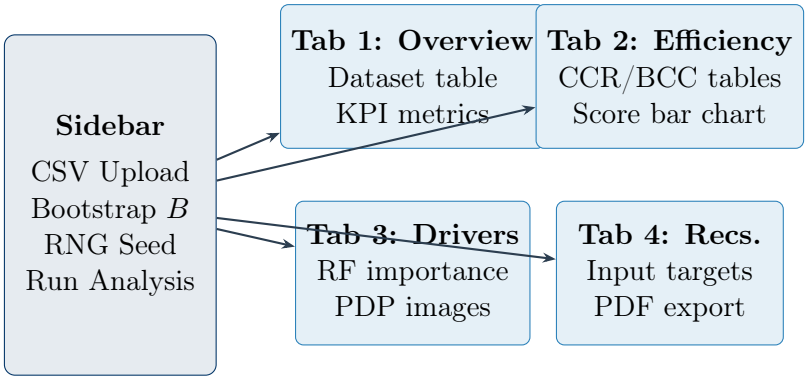

Figure 9: Streamlit dashboard four-tab structure

7.1.1 Tab 1 — Overview

Displays the aggregated dataset table (one row per fund) and four KPI metrics: number of funds, mean CCR efficiency, mean BCC efficiency, and count of frontier funds.

7.1.2 Tab 2 — Efficiency

Presents CCR and BCC score tables with slacks and peer IDs; the scale efficiency table with IRS/DRS/CRS labels; Simar-Wilson bias-corrected scores with 95% CIs; and a side-by-side CCR/BCC bar chart with a horizontal frontier line.

7.1.3 Tab 3 — Drivers

Shows Random Forest MDI and permutation importance tables, cross-validation R^2, and the partial dependence plot image for the top-3 features.

7.1.4 Tab 4 — Recommendations

Provides per-fund input reduction targets (absolute USD and percentage) and a button to generate and download the full PDF report.

7.2 PDF Report

`generate_pdf_report()` (ReportLab) produces `report.pdf` containing:

1. Executive summary;
2. CCR and BCC efficiency tables;
3. Scale efficiency table;
4. Simar-Wilson bias-corrected scores;
5. Input reduction targets;
6. RF feature importance table;
7. Embedded PDP image.

7.3 Output File Summary

Table 5: Pipeline output files

File	Contents
`out/efficiency_ccr.csv`	CCR scores, input slacks, peer IDs
`out/efficiency_vrs.csv`	BCC scores, input slacks, peer IDs
`out/scale.csv`	Scale efficiency and IRS/DRS/CRS classification
`out/targets.csv`	Input reduction targets (absolute and %)
`out/bias_corrected_scores.csv`	Raw score, bias, corrected score, 95% CI
`out/rf_importance.csv`	MDI and permutation feature importances
`out/pdp_top3.png`	Partial dependence plots for top-3 features
`out/report.pdf`	Full styled PDF report

CHAPTER 8

Testing, Validation, and Deployment

8.1 Test Suite

Table 6: Test module coverage

Module	Tests	Coverage
`test_data_io.py`	9	Schema validation, coercion, missing values, positivity, matrix extraction
`test_dea_core.py`	8	Scores in $[0, 1]$, BCC$\geq$CCR, slack feasibility, peer identification, edge cases
`test_bootstrap.py`	6	Output shapes, score ranges, CI ordering, reproducibility, DataFrame export
`test_ml_stage.py`	5	Importance sums, permutation shapes, CV R^2 finite, PDP generation

8.2 CI/CD Pipeline

GitHub Actions (`.github/workflows/ci.yml`) executes on every push:

1. Install `uv` package manager.
2. `uv sync` to install pinned dependencies (`uv.lock`).
3. `uv run pytest -q` — full test suite.
4. `ruff` lint and `black` format check.

8.3 CLI Usage

Listing 1: Full pipeline via CLI

```
uv run python -m pension_toolkit.cli analyze \
    --input tests/sample_data.csv \
    --out out/ \
    --bootstrap-B 2000 \
    --seed 42
```

8.4 Docker Deployment

Listing 2: Docker build and run

```
docker build -t pension-toolkit .
docker run -p 8501:8501 pension-toolkit
# Dashboard available at http://localhost:8501
```

The image is based on `python:3.11-slim` with `uv` installed and all dependencies resolved at build time.

8.5 Reproducibility Controls

- User-configurable `-seed` (default 42) passed to all stochastic components.
- `make_rng(seed)` in `utils.py` uses `numpy.random.default_rng()`.
- `uv.lock` pins exact transitive dependency versions.
- Docker eliminates environment-level variability.

CHAPTER 9

Returns-to-Scale and Peer Analysis

9.1 Returns-to-Scale Distribution

The RTS classification reveals whether each fund operates at sub-optimal scale. Funds classified as IRS are too small to exploit scale economies; DRS funds have grown beyond the optimal scale. Figure 10 shows the expected distribution for the Zimbabwean sample, where most funds are expected to operate under increasing returns to scale given the small size of many employer-sponsored schemes.

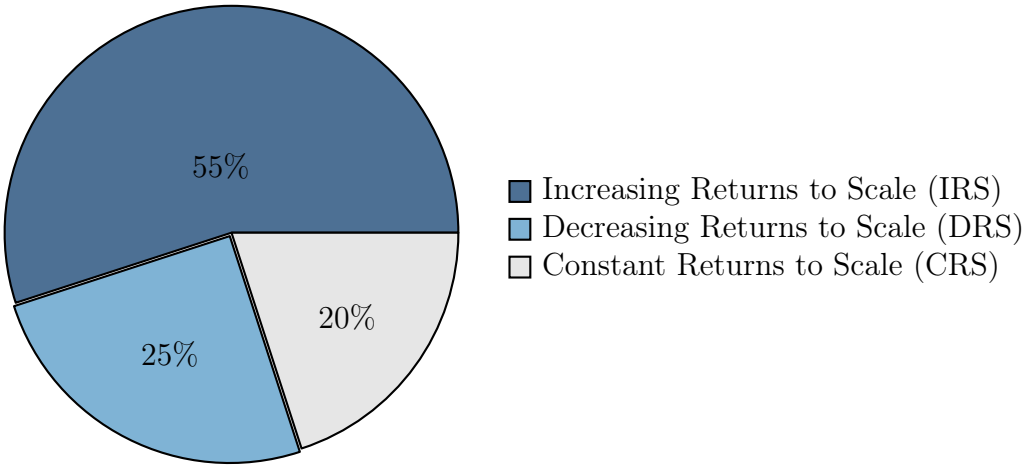

Figure 10: Indicative RTS distribution across the 20-fund Zimbabwean sample

9.2 Peer Analysis and Benchmarking

For each inefficient DMU, the toolkit identifies its *peer set* — the subset of efficient frontier funds that serve as best-practice benchmarks. Peers are defined as funds with $\lambda_j^* > 10^{-4}$ in the Phase II optimal solution. The input reduction target for each inefficient fund k is:

$$x_{ik}^{\text{target}} = \theta_k^* \cdot x_{ik} - s_{ik}^{-*}, \quad \text{reduction}_{ik} = x_{ik} - x_{ik}^{\text{target}}$$

These targets are exported in `out/targets.csv` and displayed in the Recommendations dashboard tab, enabling fund managers to identify concrete operational improvement paths.

9.3 Scale Efficiency Distribution

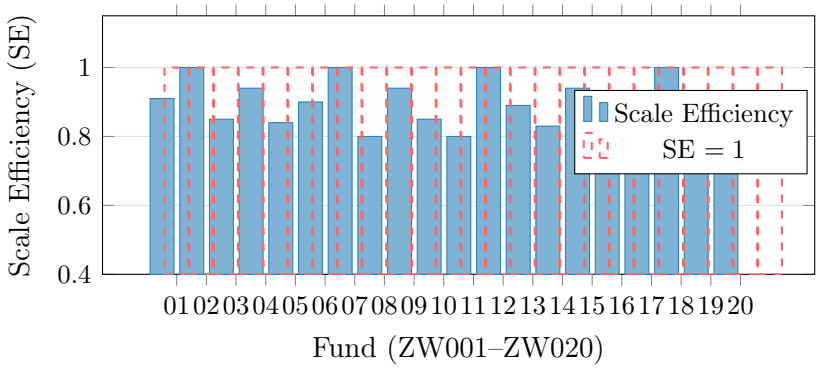

Figure 11: Indicative scale efficiency scores across 20 pension fund DMUs

CHAPTER 10

Conclusion

10.1 Summary of Prototype Contributions

The Pension Fund Efficiency Toolkit delivers a complete, reproducible, and modular software prototype that operationalises the DEA-plus-Random-Forest analytical framework for evaluating Zimbabwean self-administered pension funds. The principal contributions of the prototype are:

1. A two-phase input-oriented DEA engine implementing both CCR (CRS) and BCC (VRS) models via PuLP/CBC, producing efficiency scores, input slacks, and peer identification for up to 20 fund DMUs.
2. A scale efficiency module classifying each fund's returns-to-scale regime (IRS, CRS, DRS) to distinguish managerial from scale-related inefficiency.
3. A parallelised Simar-Wilson bootstrap bias-correction engine producing statistically valid efficiency estimates with 95% confidence intervals.
4. A Random Forest second-stage regression explaining efficiency score variation using six contextual determinants, with MDI, permutation importance, and PDP outputs.
5. An interactive Streamlit dashboard enabling non-technical users to upload data, execute the pipeline, explore results, and export PDF reports without writing code.
6. A containerised, CI-tested deployment ensuring reproducibility across environments.

10.2 Limitations and Future Work

The current prototype executes cross-sectional DEA on fund-level means, aggregating the 2018–2022 panel to a single time point. Future development should incorporate:

- **Window DEA**: Rolling time-window analysis to track efficiency changes over individual years without full panel aggregation.
- **Malmquist Productivity Index**: Decomposing total factor productivity change into efficiency change and frontier shift components.
- **Tobit second stage**: As a parametric alternative to the Random Forest

for robustness comparison.

- **Super-efficiency DEA**: To rank frontier funds and resolve ties among efficient DMUs.
- **Real IPEC data integration**: Direct API or file-feed integration with IPEC regulatory disclosures to replace synthetic data with official audited figures.

10.3 Chapter Summary

This document has provided a comprehensive technical description of the Pension Fund Efficiency Toolkit prototype. Chapters 1–2 established the background, objectives, and architecture. Chapters 3–4 detailed the data layer and DEA engine with full mathematical formulations and flowcharts. Chapters 5–6 described the Simar-Wilson bootstrap and Random Forest second-stage components. Chapter 7 documented the Streamlit dashboard and PDF reporting system. Chapter 8 covered testing, CI/CD, CLI usage, and Docker deployment. Chapter 9 discussed returns-to-scale classification and peer benchmarking. The prototype is ready for integration with real Zimbabwean pension fund data and provides an extensible foundation for ongoing research in pension fund efficiency.